

KOTLIN: ОСНОВЫ ЯЗЫКА

Проверка типов и сравнение

is / !is — проверка типа

Проверяет тип объекта с автоматическим smart cast:

```
val obj: Any = "hello"

if (obj is String) {
    println(obj.length) // obj уже String – smart cast
}

if (obj !is String) {
    println("Не строка")
}
```

== / != — сравнение по значению

Вызывает equals():

```
val list1 = listOf(1, 2, 3)
val list2 = listOf(1, 2, 3)

println(list1 == list2) // true – содержимое одинаковое
```

=== / !== — сравнение по ссылке

Проверяет, один ли это объект в памяти:

```
val a = listOf(1, 2, 3)
val b = listOf(1, 2, 3)
val c = a

println(a === b) // false – разные объекты
println(a === c) // true – одна и та же ссылка
```

Data class

Что генерирует data class

```
data class User(val name: String, val age: Int)
```

Автоматически генерирует: - equals() — сравнение по полям - hashCode() — хеш на основе полей - toString() — User(name=Alice, age=25) - copy() — копирование с изменением полей - component1(), component2() — для деструктуризации

Зачем нужен data class для коллекций

Без data class — объекты сравниваются по ссылке:

```
class User(val id: Int, val name: String)

val user1 = User(1, "Alice")
val user2 = User(1, "Alice")

val set = setOf(user1)
println(set.contains(user2)) // false! Разные объекты в памяти
```

C data class — объекты сравниваются по содержимому:

```
data class User(val id: Int, val name: String)

val user1 = User(1, "Alice")
val user2 = User(1, "Alice")

val set = setOf(user1)
println(set.contains(user2)) // true! Содержимое одинаковое
```

copy() — копирование с изменениями

```
data class User(val name: String, val age: Int)

val alice = User("Alice", 25)
val olderAlice = alice.copy(age = 30)
// User(name=Alice, age=30)
```

Важно: copy() создаёт shallow copy — вложенные объекты не копируются:

```
data class User(val name: String, val address: Address)
data class Address(var city: String)

val user1 = User("Alice", Address("NYC"))
val user2 = user1.copy()

user2.address.city = "LA"
println(user1.address.city) // "LA" — изменилось и у user1!
```

equals/hashCode используют только primary constructor

```
data class Person(val name: String) {
    var age: Int = 0 // НЕ участвует в equals/hashCode!
}
```

```
val p1 = Person("John").apply { age = 25 }
val p2 = Person("John").apply { age = 30 }

println(p1 == p2) // true – age не учитывается
```

Деструктуризация

Разбирает объект на составные части через componentN() функции.

Pair и Triple

```
val pair = "Alice" to 25
val (name, age) = pair

val triple = Triple("Alice", 25, "Dev")
val (n, a, job) = triple
```

Data class

```
data class User(val name: String, val age: Int)

val user = User("Alice", 25)
val (name, age) = user
```

В циклах по Map

```
val map = mapOf("a" to 1, "b" to 2)

for ((key, value) in map) {
    println("$key = $value")
}

map.forEach { (key, value) ->
    println("$key = $value")
}
```

Пропуск ненужных компонентов

```
val (name, _) = user // age не нужен
```

Иерархия коллекций (схема)

```
Iterable<T>
├── Collection<T>
│   ├── List<T>
│   └── Set<T>
```

```

└─ MutableIterable<T>
    └─ MutableCollection<T>
        └─ MutableList<T>
            └─ MutableSet<T>

```

Map<K,V> (отдельно, не Iterable!)

```

└─ MutableMap<K,V>

```

Ключевое: - List, Set, Map — read-only интерфейсы - MutableList, MutableSet, MutableMap — с методами изменения - Map не наследует Iterable, но можно итерировать по entries, keys, values

Контракт equals и hashCode

Правило: если `a == b` (equals), то `a.hashCode() == b.hashCode()`

Нарушение ломает HashSet и HashMap:

```

class BadUser(val id: Int) {
    override fun equals(other: Any?) =
        other is BadUser && id == other.id
    // hashCode НЕ переопределён – ОШИБКА!
}

val set = hashSetOf(BadUser(1))
println(set.contains(BadUser(1))) // false! Разные hashCode → разные bucket'

```

Решение: всегда используй data class или переопределяй оба метода.

Перегрузка операторов

Kotlin позволяет определять поведение операторов через operator fun.

Основные операторы

Оператор	Функция	Пример
<code>a + b</code>	<code>a.plus(b)</code>	Сложение
<code>a - b</code>	<code>a.minus(b)</code>	Вычитание
<code>a * b</code>	<code>a.times(b)</code>	Умножение
<code>a / b</code>	<code>a.div(b)</code>	Деление
<code>a == b</code>	<code>a.equals(b)</code>	Равенство
<code>a < b</code>	<code>a.compareTo(b) < 0</code>	Сравнение

Оператор	Функция	Пример
<code>a in b</code>	<code>b.contains(a)</code>	Принадлежность
<code>a[i]</code>	<code>a.get(i)</code>	Доступ по индексу
<code>a[i] = x</code>	<code>a.set(i, x)</code>	Присваивание

Пример: Vector

```
data class Vector(val x: Int, val y: Int) {
    operator fun plus(other: Vector) = Vector(x + other.x, y + other.y)
    operator fun minus(other: Vector) = Vector(x - other.x, y - other.y)
}
```

```
val v1 = Vector(1, 2)
val v2 = Vector(3, 4)
println(v1 + v2) // Vector(4, 6)
```

Пример: compareTo для сравнений

```
data class Version(val major: Int, val minor: Int) : Comparable<Version> {
    override fun compareTo(other: Version): Int {
        return when {
            major != other.major -> major - other.major
            else -> minor - other.minor
        }
    }
}
```

```
val v1 = Version(1, 2)
val v2 = Version(2, 0)
println(v1 < v2) // true
```

Полезные мелочи

Elvis оператор ?:

```
val name: String? = null
val result = name ?: "default" // "default"
```

let / also / apply / run

// let – преобразование

```
val length = name?.let { it.length }
```

// also – side effect, возвращает исходный объект

```
val user = User("Alice").also { println("Created: $it") }
```

// apply – настройка объекта

```
val user = User().apply {
```

```
    name = "Alice"  
    age = 25  
}
```

// run – выполнение блока с результатом

```
val result = user.run { "$name is $age years old" }
```

Ranges

```
val inRange = 5 in 1..10 // true
```

```
val chars = 'a'..'z'
```

```
for (i in 1..5) { } // 1, 2, 3, 4, 5
```

```
for (i in 1 until 5) { } // 1, 2, 3, 4
```

```
for (i in 5 downTo 1) { } // 5, 4, 3, 2, 1
```

```
for (i in 1..10 step 2) { } // 1, 3, 5, 7, 9
```